

# 推論専用公開モデルに対する分散ファインチューニングの実行正当性検証

— OpenDDE 拡散モジュールを 4 ノード 16 GPU で 30 エポック学習した記録 —

松澤 拓海

2026 年 8 月 15 日

## Abstract

マルチノード分散学習では、計算資源が確保されているにもかかわらず一部のランクしか計算に参加していないという故障が起こりうる。この故障は例外を送出せず、ジョブスケジューラ上でも正常なジョブと区別がつかないため、通常の実行ログからは検出できない。本研究は、全ランクが自身のホスト名と GPU の一意識別子を報告し、それを NCCL の集団通信で集約するという検証機構を実装し、その集約結果の出力自体をノード間通信成立の証拠として用いる。検証対象として、推論専用ビルドとして公開されている生体分子構造予測モデル OpenDDE を選び、技術報告書に記載されたハイパーパラメータから学習ループを再構成した。理化学研究所 RIKYU クラスターの 4 ノード 16 GPU (NVIDIA GB200) 上で 30 エポック、総計 960 ステップ、1411 秒の分散学習を実行し、16 ランクすべてが 4 ノードに均等配置され、同数のステップを実行したことを確認した。一方、FoldBench の抗体-抗原インターフェース予測における DockQ 成功率は事前学習済みモデルと微調整済みモデルで同一 (ともに 10.53%、19 インターフェース中 2 件) であり、平均 DockQ はむしろ 0.0844 から 0.0693 へ低下した。学習損失も 30 エポックを通じて傾向を示さなかった。これらは実験前に文書化した予測と一致する。本研究の主張は精度向上ではなく、分散実行の正当性を検証可能にする手法の提示にある。

## 1 はじめに

大規模モデルの学習では複数ノードにまたがる分散実行が前提となる。その際に問題となるのは、計算が明示的に失敗する場合ではなく、失敗しないまま誤った結果を出す場合である。

具体例を挙げる。分散学習の実行基盤が環境変数 `RANK` や `WORLD_SIZE` を静的な単一値として全プロセスに配布した場合、16 個のプロセスはいずれも「自分は単独プロセスである」と認識する。勾配同期は行われず、実質的に 16 個の独立した学習が走るが、例外は送出不されず、ジョブは正常終了し、スケジューラ上は 4 ノードを確保した正常なジョブとして記録される。実行ログを開いても、学習ステップと損失が出力されている以上、異常を示す手がかりは存在しない。

本研究はこの種の故障を検出可能にすることを目的とする。中心となる考え方は単純である。全ランクに自身のホスト名と GPU の一意識別子を報告させ、その報告を集団通信で集約する。集約結果が出力されたという事実自体が、ノード間でデータが実際に流れたことの証拠となる。確保されたノード数ではなく、報告に現れた相異なるホスト名の個数を数えることで、「確保されている」と「計算している」を分離できる。

検証対象として OpenDDE [5] を用いた。これは Apache-2.0 で公開された全原子生体分子基盤モデルであり、抗体-抗原共フォールディングにおいて既存のオープンモデルを上回る精度が報告されている。選定理由は後述するが、公開版が推論専用ビルドであるため、分散学習を行うには学習ループ自体を外部から構成する必要がある、検証機構を組み込む余地があった点大きい。

## 2 関連研究

### 2.1 全原子構造予測モデル

AlphaFold3 [1] は、タンパク質・核酸・低分子・イオンを含む複合体の原子レベル同時モデリングを実現した。その拡散モジュールは、ノイズを加えた座標から真の座標を復元する学習によって訓練される。OpenDDE [5] はこの枠組みを踏襲しつつ Pairformer の隠れ次元を 128 から 384 に拡大し、原子レベルの潜在推論ブロックを導入したモデルである。

### 2.2 評価ベンチマーク

FoldBench [7] は 1,522 個の生物学的アセンブリを 9 種のタスクに分類したベンチマークである。本研究では抗体-抗原インターフェース予測を用いる。インターフェース予測の品質は DockQ [2] で測る。DockQ は CAPRI の分類基準を連続値化した指標で、0.23 を超えると acceptable、0.49 を超えると medium、0.80 を超えると high と分類される。算出には OpenStructure [6] の `compare_structures` を用いた。

### 2.3 分散学習の検証

PyTorch の DistributedDataParallel [4] は、各プロセスがモデルの複製を保持し、逆伝播のたびに勾配を集団通信で平均する方式をとる。本研究が扱う故障は、この枠組みが正しく初期化されなかった場合に生じる。分散学習の性能評価は広く行われている一方、実行が意図した並列度で行われたことの事後検証に焦点を当てた報告は少ない。

## 3 背景

### 3.1 公開版 OpenDDE には学習コードが存在しない

OpenDDE の技術報告書は「学習コード、推論パイプライン、チェックポイント、ベンチマークを公開する」と述べている [5]。しかし本研究の時点で公開リポジトリを調査した結果、学習ループ、損失関数、オプティマイザのいずれも存在しなかった。ドキュメント内にも「推論専用ビルド (inference-only build)」という記述があり、公開されているのは推論経路のみである。

したがって本研究における学習は、公開コードの再現ではなく、技術報告書に記載されたハイパーパラメータからの再構成である。この区別は重要であり、本稿で報告する学習の挙動は OpenDDE 開発陣の学習手順と一致する保証がない。

### 3.2 再構成に利用できた要素

一方で、推論専用ビルドにも学習に転用できる要素が残されていた。第一に、拡散モジュールの `forward` が「ノイズ付き座標とノイズ強度を受け取り、デノイズ済み座標を返す」単一ステップのデノイザとして公開されている。これは拡散学習の損失を接続する対象そのものである。第二に、実装内の in-place 最適化はすべて勾配が有効な場合に自動的に無効化される設計になっており、逆伝播を妨げない。第三に、カスタム CUDA LayerNorm は `backward` を実装した `torch.autograd.Function` として定義されている。

ただしトランク (入力埋め込み、MSA モジュール、テンプレート埋め込み、Pairformer) は `torch.set_grad_enabled(False)` で囲まれており、勾配を受け取れない。本研究ではこれを制約ではなく設計として受け入れ、拡散モジュールのみを学習対象とした。

## 4 手法

### 4.1 学習目的関数の再構成

拡散学習の損失は EDM [3] の定式化に従う。ノイズ強度  $\sigma$  を対数正規分布から標本化する:

$$\sigma = \sigma_{\text{data}} \exp(p_{\text{mean}} + p_{\text{std}} \cdot n), \quad n \sim \mathcal{N}(0, 1) \quad (1)$$

ここで  $p_{\text{mean}} = -1.2$ 、 $p_{\text{std}} = 1.5$ 、 $\sigma_{\text{data}} = 16.0$  は OpenDDE 技術報告書 Table 1(b) に記載された値である [5]。

正解座標  $x_{\text{GT}}$  に対しノイズを加え、デノイザ  $D_\theta$  で復元し、AlphaFold3 補遺 3.7.1 の重み付き平均二乗誤差 [1] を損失とする:

$$\mathcal{L} = \mathbb{E}_\sigma \left[ \frac{\sigma^2 + \sigma_{\text{data}}^2}{(\sigma \cdot \sigma_{\text{data}})^2} \cdot \frac{1}{3N} \sum_i w_i \|D_\theta(x_{\text{noisy}}, \sigma)_i - \hat{x}_{\text{GT}, i}\|^2 \right] \quad (2)$$

$\hat{x}_{\text{GT}}$  は予測構造に対して重み付き剛体重ね合わせを施した正解座標であり、剛体運動の自由度を損失から除去する。 $w_i$  は原子種別ごとの重みである。

### 4.2 正解座標のモデル原子順序への整列

推論専用ビルドは正解座標を生成する経路を持たない。特徴量化が出力するのは CCD の理想化配座であって実験座標ではなく、ラベル用のスキーマは定義だけが残された未使用の状態にある。したがって教師信号の構成は自前で行う必要がある。

本研究では、各エントリに同梱される FASTA (鎖ごとの配列) と NPZ (実験座標) から教師信号を構成した。FASTA のヘッダに記された鎖 ID をそのまま入力 JSON に指定することで、モデルが構築する原子配列と正解座標側の鎖命名が構成上一致する。両者を (鎖 ID, 残基番号, 原子名) の三つ組で結合し、一致しない原子および NaN として格納されている未観測原子はマスクした。

なお、同梱される mmCIF は使用していない。前処理済み mmCIF には `_entity_poly` カテゴリが欠落しており、OpenDDE の mmCIF パーサが全ポリマーをリガンドと誤認して原子単位に分解するためである。この経路では結合率が 0% となった。

整列の正しさは幾何的に検証した。8 エントリにおいて、隣接原子間距離の中央値はすべて 1.52–1.53 Å であった。これは C–C および C–N 共有結合長に一致し、原子が正しい順序で対応付いていなければ得られない値である。

### 4.3 分散実行の検証機構

各ランクは学習開始前に、自身のホスト名、GPU の製品名、GPU の一意識別子 (UUID) を 1 行の JSON として出力する。続いてこれらのレコードを `all_gather_object` で集約し、ランク 0 が集計結果を出力する。集計には次の 3 つが含まれる: 相異なるホスト名の個数、相異なる GPU UUID の個数、ホストごとのランク数である。

期待ノード数を設定として与え、集計されたホスト数これが下回る場合は実行を失敗として扱う。したがって、16 ランクが 1 ノードに集中していた場合、成功として記録されることはない。

さらに各ランクは互いに素なデータ断片を担当する。全ランクが同一のバッチを計算しているだけという状態は、この構成では発生しない。

## 5 実験設定

### 5.1 計算環境

理化学研究所 RIKYU クラスタを用いた。1 ノードあたり NVIDIA GB200 を 4 基、CPU 144 コア、メモリ 1.72 TB を備え、アーキテクチャは aarch64、ジョブスケジューラは Slurm である。実行は 4 ノード 16 GPU を占有して行った。

## 5.2 学習データ

OpenFold3 の PDB 学習サブセット (1,000 エントリ) から、タンパク質のみで構成され、全鎖が異なる配列を持ち、非標準残基を含まない 2 鎖複合体を選定した。全鎖異配列を条件としたのは、同一配列の鎖が複数ある場合に鎖の対応付けが一意に定まらず、OpenDDE が対称性を扱う特徴量を持たない以上、任意のラベル付けに向けてモデルを学習させることになるためである。

条件を満たす 30 件のうち、原子数 4,000 を超える 3 件を除外し、27 件を用いた。除外は再現可能な基準に基づく。この上限を設けない場合、構造トークンのリファイナが単一のテンソルに 144 GiB を要求し、実行が確保したランクの担当データに依存して成否が変わる。

なお、これらのエントリの登録日は 1976 年から 2021 年 9 月であり、OpenDDE の学習データカットオフ (2021 年 9 月) の内側にある。すなわちモデルにとって既知の分布であり、新しい情報を含まない。

## 5.3 学習設定

学習率  $1 \times 10^{-4}$  の AdamW、勾配クリッピング閾値 10.0、トランクの再循環回数 4、1 ステップあたりのノイズ標本数 4 とし、30 エポック実行した。学習対象は拡散モジュールの 203.3 M パラメータであり、モデル全体 655.8 M のうち残りは凍結した。

## 5.4 評価設定

FoldBench の抗体-抗原インターフェースから 16 ターゲットを対象とし、原子数 8,000 を超える 2 件を除外して 14 ターゲット (19 インターフェース) を採点した。拡散サンプリングは 20 ステップ、1 ターゲットあたり 1 サンプル、乱数種は 42 とした。

推論では MSA およびテンプレート検索を行っていない。これは絶対精度に対する大きな不利であるが、事前学習済みモデルと微調整済みモデルの双方に同一条件を課すため、両者の比較においては相殺される。したがって本稿の絶対値を **FoldBench** の公開リーダーボードと比較してはならない。

# 6 結果

## 6.1 分散実行は正しく行われた

30 エポックの学習は 1411 秒で完了し、検証は成功と判定された。集計結果を表 1 に示す。

Table 1: 分散実行の集計結果。集計自体が `all_gather_object` の出力であり、その存在がノード間通信の成立を示す。

| 項目              | 値                          |
|-----------------|----------------------------|
| 相異なるホスト数        | 4 (c050, c054, c055, c056) |
| ホストごとのランク数      | 各 4                        |
| 相異なる GPU UUID 数 | 16                         |
| 総ランク数           | 16                         |
| 総ステップ数          | 960 (= 16 ランク × 60 ステップ)   |
| 実行時間            | 1411 秒                     |
| 判定              | PASS                       |

GPU UUID が 16 個すべて相異なることは、物理的に別々の 16 基の GPU が使用されたことを意味する。総ステップ数が 960 であり、ランクあたり 60 ステップで均等であることは、全ランクが同量の計算を担ったことを示す。

学習中、勾配ノルムは全ランクで一致していた (エポック 3 で 0.056、エポック 14 で 0.028)。これは DDP が勾配を平均していることの直接的な帰結であり、損失がランクごとに異なる (各ランクが異なる構造を担当するため) のと対照的である。

## 6.2 学習損失は傾向を示さなかった

図 1 にエポックごとの損失推移を示す。値は 0.0373 から 0.0556 の範囲を上下するのみで、線形回帰はほぼ水平である。初回エポックの 0.0498 に対し最終エポックは 0.0469 であり、実質的な変化はない。

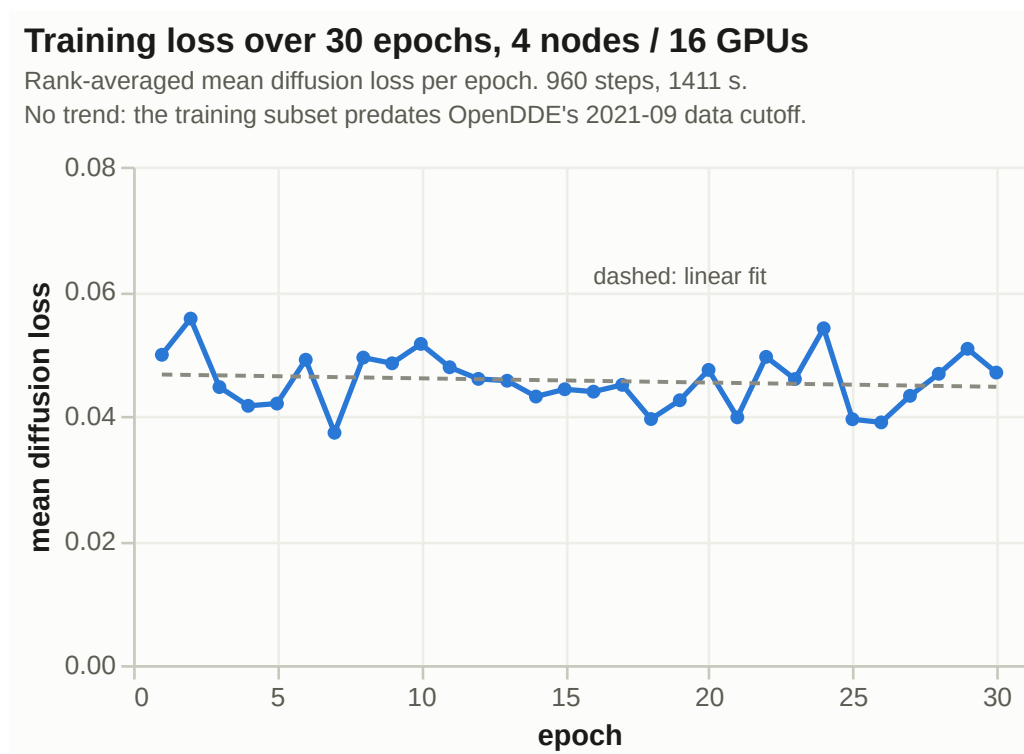


Figure 1: 30 エポックの学習損失推移。全ランク平均。破線は線形回帰。3 エポックの実行では「揺らぎかもしれない」としか言えないが、30 エポックでは傾向の不在を主張できる。

## 6.3 DockQ は改善しなかった

FoldBench 抗体-抗原インターフェースにおける評価結果を表 2 に示す。

Table 2: FoldBench 抗体-抗原インターフェース (19 インターフェース) の評価結果。MSA・テンプレート検索なし。

| 指標                     | 事前学習済み        | 微調整済み (30 エポック) |
|------------------------|---------------|-----------------|
| DockQ 成功率 ( $> 0.23$ ) | 10.53% (2/19) | 10.53% (2/19)   |
| DockQ 平均               | 0.0844        | 0.0693          |
| DockQ 中央値              | 0.039         | 0.034           |
| medium 率 ( $> 0.49$ )  | 0%            | 0%              |
| IDDT 平均                | 0.660         | 0.643           |

主要指標である DockQ 成功率は両者で完全に一致した。平均 DockQ および IDDT 平均はいずれもわずかに低下している。図 2 の累積分布では 2 本の曲線がほぼ重なり、どちらも acceptable 閾値の手前で落ちきっている。

インターフェースごとに見ると (図 3)、変化は両方向に散らばっている。改善した例 (8fdo: 0.068  $\rightarrow$  0.300、8hxq: 0.176  $\rightarrow$  0.283) と悪化した例 (8fdd-1: 0.242  $\rightarrow$  0.058、8ux6-1: 0.317  $\rightarrow$  0.174) が併存し、系統的な傾向は認められない。

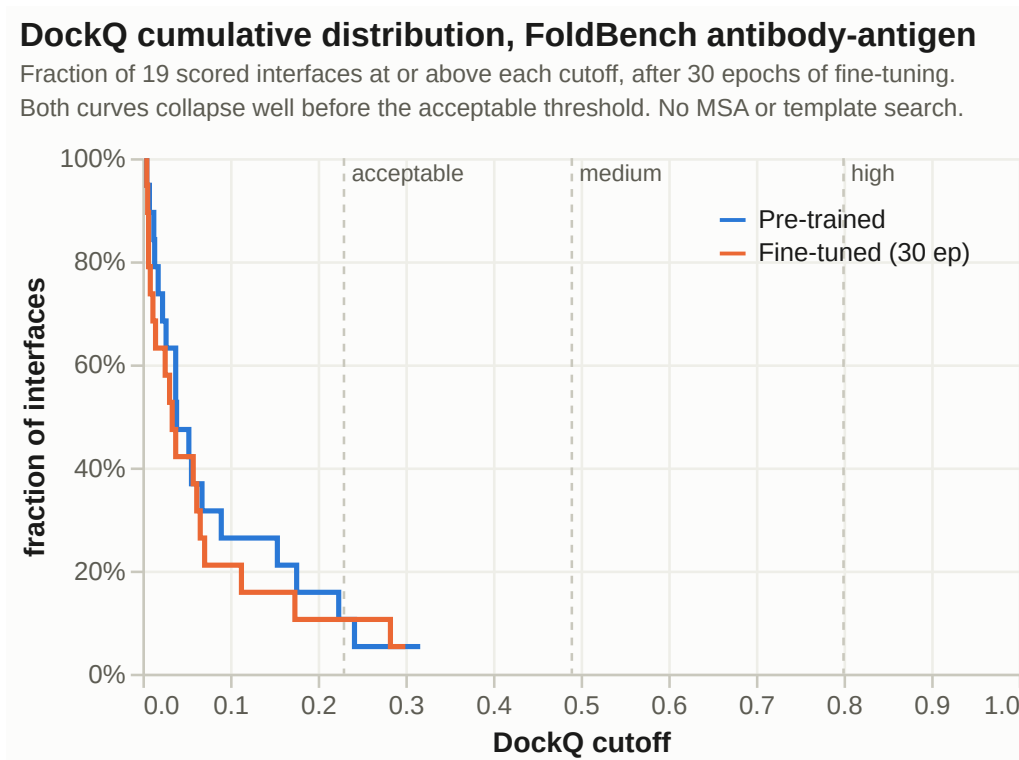


Figure 2: DockQ の累積分布。横軸の各閾値を超えるインターフェースの割合。

## 7 考察

### 7.1 予測との一致

本研究では実験実施前に、分散実行は正しく動作する一方 DockQ は改善しないかわずかに低下すると予測し、その理由を文書として記録した。結果は予測と一致した。理由として挙げた「学習データが事前学習分布の内側にある」という点は、学習損失が 30 エポックにわたり傾向を示さなかったことから支持される。

予測を事前に記録したのは、結果を得てから解釈を都合よく変更することを防ぐためである。本研究のように否定的な結果が予想される場合、この手続きの価値は特に大きい。

### 7.2 エポック数を伸ばして得られたもの

先行して実施した 3 エポックの実行 (96 ステップ、166 秒) と比較すると、本実行は 10 倍の規模である。この延長によって得られた情報は 2 つある。

第一に、損失の傾向の不在を主張できるようになった。3 エポックでは「揺らいでいるだけかもしれない」としか言えない。30 点あることで、線形回帰が水平であることを図として提示できる。

第二に、長時間実行における分散実行の安定性が確認された。166 秒では、通信が徐々に劣化する種の異常は発現する時間すら与えられていない。41 分にわたり勾配同期が維持されたことは、より強い主張である。

## 8 限界

本研究には次の限界がある。

学習コードは公開版の再現ではない。OpenDDE の公開リポジトリに学習コードが存在しないため、本研究の学習ループは技術報告書の記述からの再構成である。開発陣の実際の学習手順と一致する保証はない。

### Per-interface DockQ, pre-trained vs 30-epoch fine-tuned

Each of 19 scored interfaces under both checkpoints, sorted by pre-trained score.

Dashed line: acceptable threshold (DockQ 0.23). Two interfaces clear it under each model.

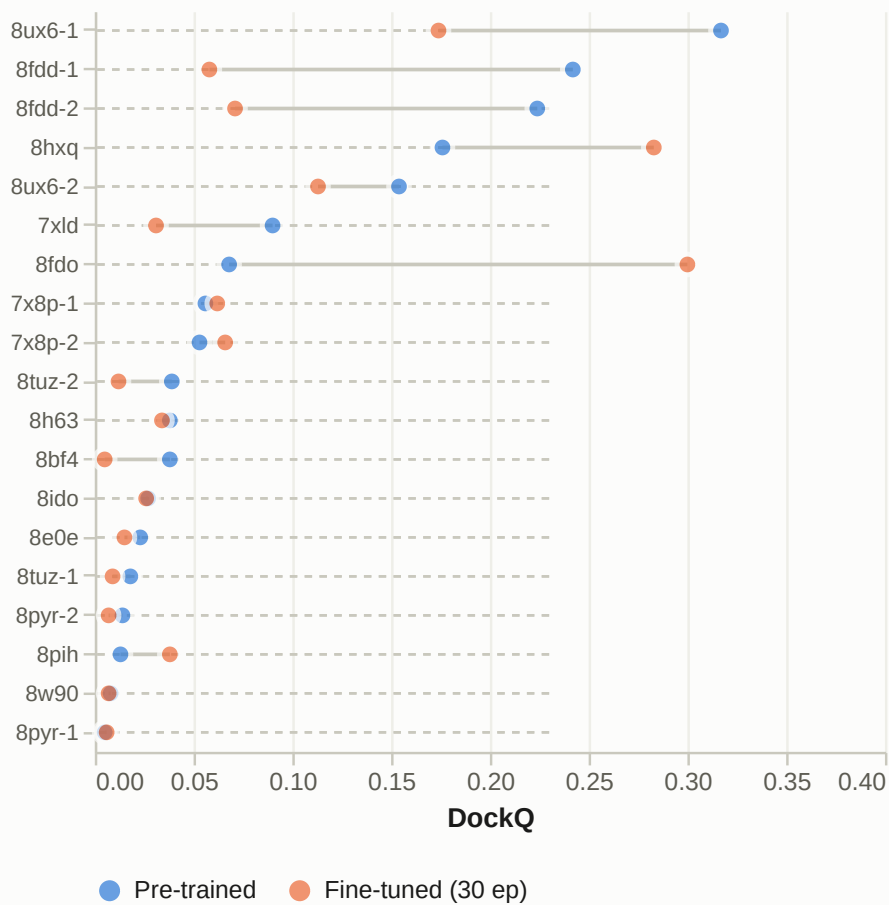


Figure 3: インターフェースごとの DockQ。各行が 1 インターフェースで、両モデルの値を線で結んでいる。

**InfiniBand** を使用していない。ジョブコンテナに `libibverbs` が含まれないため、NCCL は IPoIB 上の TCP ソケットにフォールバックしている。通信は成立するが、GB200 の相互接続性能は発揮されていない。本研究は分散実行の正当性を扱うものであり、性能については何ら主張しない。

スケーリングを測定していない。1、4、8、16 GPU での処理時間比較を行っていないため、ノード数を増やした際の効率是不明である。

評価規模が小さい。19 インターフェースでの比較であり、DockQ 成功率における 1 件の差は約 5 ポイントに相当する。本研究の結論は「改善が観測されなかった」ことであり、「改善しないことが示された」ではない。

学習データが小規模である。27 件という規模は、655 M パラメータのモデルに対して意味のある更新を与えるには小さい。条件を緩めずに手元のサブセットから得られる上限は 54 件であった。

## 9 結論

推論専用ビルドとして公開されたモデルに対し、技術報告書の記述から学習ループを再構成し、4 ノード 16 GPU の分散環境で 30 エポック、960 ステップの学習を実行した。16 ランクすべてが 4 ノードに均等配置され、相異なる 16 基の GPU 上で同数のステップを実行したことを、集団通信による集約結果として確認した。

精度は改善しなかった。DockQ 成功率は事前学習済みと同一であり、平均 DockQ は低下した。学習損失も傾向を示さなかった。これらは実験前に記録した予測と一致し、学習データが事前学習分布の内側にあるという説明と整合する。

本研究の寄与は、「確保されている」と「計算している」を分離して記録するという単純な機構にある。この機構がなければ、16 ランクが 1 ノードに集中した実行も、勾配同期が行われない実行も、成功として記録されうる。実際、本研究の実装過程では、コンテナのエントリポイントが何も実行しないまま 42 秒で正常終了し、ジョブが成功として記録される事象が発生した。ログを開いて初めて発覚したものである。分散実行の検証は、実行基盤の報告する成否とは独立に行われる必要がある。

## References

- [1] Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J Ballard, and Joshua Bambrick. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630:493–500, 2024.
- [2] Sankar Basu and Bjorn Wallner. Dockq: A quality measure for protein-protein docking models. *PLoS ONE*, 2016.
- [3] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *NeurIPS*, 2022.
- [4] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: Experiences on accelerating data parallel training. *VLDB*, 2020.
- [5] Aureka AI OpenDDE project. Folding, reasoning, and scaling with open-source drug discovery engine. *arXiv preprint arXiv:2607.03787*, 2026.
- [6] Gerardo Studer, Christine Rempfer, Andrew M Waterhouse, Rafal Gumienny, Juergen Haas, and Torsten Schwede. Modeling biomolecular complexes with openstructure. *Scientific Reports*, 2020.



- [7] Sheng Xu, Qili Feng, Lifeng Qiao, Hao Wu, Tao Shen, Yu Cheng, Shuangjia Zheng, and Siqi Sun. Benchmarking all-atom biomolecular structure prediction with foldbench. *Nature Communications*, 2025.